Inteligență Artificială

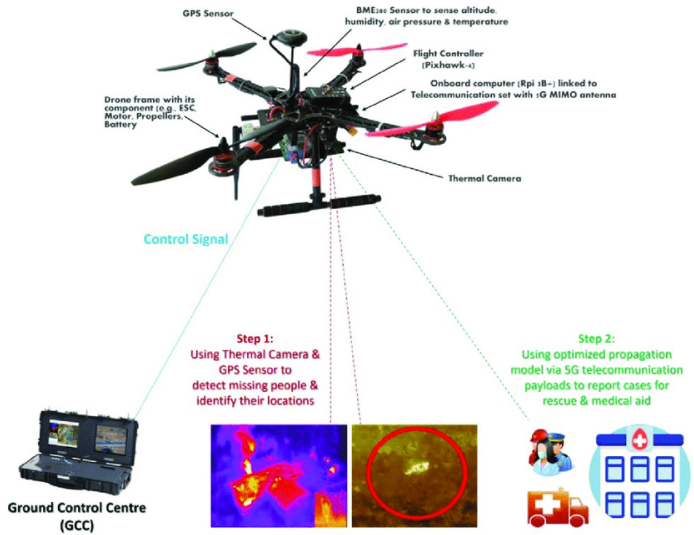
Agenți Inteligenți

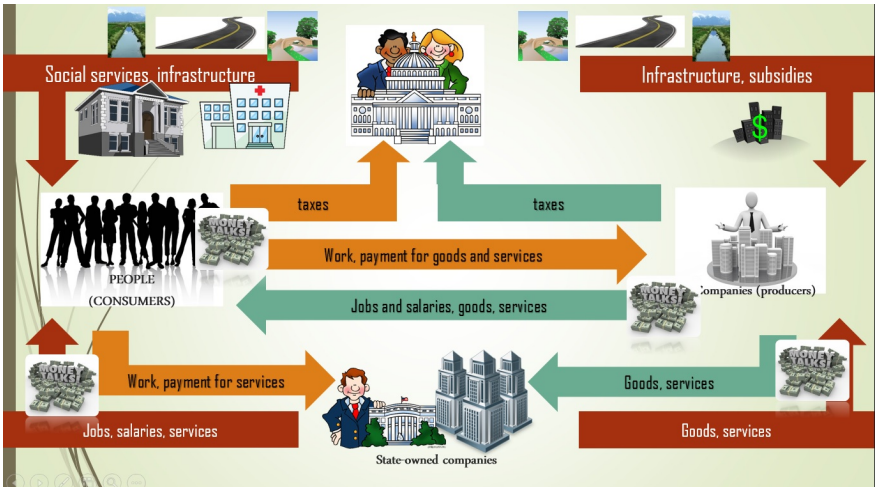
Slides: Alexandru Sorici

- Definiție
- Caracteristici Agenți
- Caracteristici Mediu
- Arhitecturi Agenți



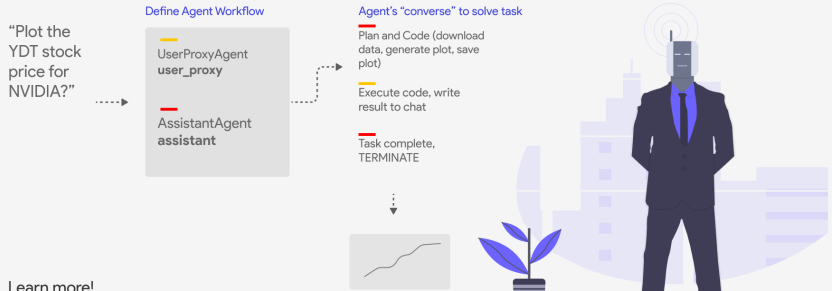

14:32
Catalyst LE
AlphaStar
LiquidTLO
SUPPLY
MINERALS
GAS
WORKERS
ARMY
APM
PRODUCTION


GPS Sensor
BME280 Sensor to sense altitude, humidity, air pressure & temperature
Flight Controller (Pixhawk-4)
Onboard computer (Rpi 3B+) linked to Telecommunication set with 5G MIMO antenna
Drone frame with its component (e.g. ESC, Motor, Propellers, Battery
Thermal Camera
Control Signal
Step 1:
Using Thermal Camera & GPS Sensor to detect missing people & identify their locations
Step 2:
Using optimized propagation model via 5G telecommunication payloads to report cases for rescue & medical aid
Ground Control Centre (GCC)


Social services, infrastructure
Infrastructure, subsidies
taxes
taxes
PEOPLE
(CONSUMERS)
Work, payment for goods and services
Jobs and salaries, goods, services
Work, payment for services
Goods, services
Jobs, salaries, services
Goods, services
State-owned companies

Introduction to AutoGen

A Framework for Building Multi-Agent Applications

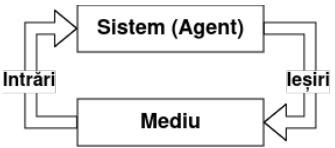


Learn more!
newsletter.victordibia.com

- "De cele mai multe ori, oamenii folosesc termenul **agent** pentru a referi o entitate care funcționează permanent și autonom într-un mediu în care există alte procese și/sau alți agenți"
- "Un **agent** este o entitate care percepe mediul în care se afla și acționează asupra acestuia"

Un **agent** este un sistem (software sau hardware) cu următoarele proprietăți:

- Autonomie: agenții operează fără intervenția directă a utilizatorului și au un anumit control asupra acțiunilor și stărilor lor
- Reactivitate: agenții percep mediul și reacționează corespunzător al schimbările din acesta
- Pro-activitate: agenții, pe lângă reacția la schimbările din mediu, sunt capabili sa urmărească execuția scopurilor și să acționeze independent
- Abilități sociale: agenții interacționează cu alți agenți sau cu utilizatorul pe baza unui limbaj de comunicare



- **Funcționalitate orientată pe obiectiv**: agenții au un **scop** pe care *vor* (intenționalitate) sa-l împlinească
  - Un agent operează pentru un *utilizator* sau un *program*

- **Percepția și acțiunea asupra mediului**:
  - Diferența între comportament reactiv și pro-activ
  - Comportament **reactiv**: agentul *reacționează* la un *eveniment* / *stimul* din mediu: e.g. un termostat este un agent reactiv
  - Comportament **pro-activ**: agenții *generează ei înșiși obiective*. Nu sunt influențați doar de evenimente din mediu, e.g.: un NPC în Starcraft, o dronă de search-and-rescue, asistent digital (Google Assistant, Alexa)

- **Caracter social**: agenții se pot constitui într-un **sistem organizațional** mai amplu, în care agenții *comunică prin mesaje*, și în care se definesc *structuri organizaționale*, *roluri*, *obiective* și *norme de interacțiune*: e.g. o linie de asamblare flexibilă, organizarea ierarhică într-un sistem robotizat de intervenție în caz de dezastru

- **Mobilitate**: în termeni fizici (pentru agenți "embodied"), în termeni de schimbare a rolurilor sau a comportamentului, în termeni de schimbare a suportului hardware pe care rulează (serializare –> deserializare)

- **Raționament / comportament "inteligent"**: agenții care sunt pro-activi vor acționa astfel încât să-și îndeplinească obiectivele, nu să le împiedice, și vor menține, de regulă, o noțiune de utilitate a situației lor actuale într-un mediu

- **Învățare / Adaptare**: interacțiuni repetate cu un mediu servesc agenților raționali, pentru a învăța cum să *maximizeze utilitatea* pe care o pot cumula în mediu (a optimiza interacțiunea cu mediul și cu alți agenți)

Sisteme Multi-Agent (SMA)

- Sistem care implică existența mai multor agenți în același mediu
- În sisteme pot apărea interacțiuni (la nivel de comunicare, organizare, acțiuni asupra mediului) de:
  - coordonare / colaborare: agenții își împart roluri într-un **obiectiv comun**: e.g. jocuri de echipă, sisteme industriale
  - competitive: agenții urmăresc un **obiectiv individual** ce poate fi în conflict cu obiectivele celorlalți agenți: e.g. jocuri zero-sum, competiție pentru resurse
- Într-un SMA agenții trebuie să dezvolte **limbaje** și **protocoale de interacțiune** (e.g. negociere, formarea de coaliții, asignare de task-uri, votare)
- Într-un SMA apar **structuri organizaționale** descrise în termeni de:
  - Grupuri și roluri într-un grup
  - Misiuni per rol, per grup
  - Norme de interacțiune (e.g. interdicții, responsabilități, prescripții)

Mediul agenților

- **Accesibil vs. inaccesibil**:
  - Accesibil: agenții obțin informații complete și exacte despre mediu – foarte rar în practică (e.g. în jocuri sau în sisteme de control industrial)
  - Inaccesibil: mediul poate fi perceput doar parțial, prin intermediul unor observații
- **Determinist vs. non-determinist**:
  - Determinist: toate acțiunile asupra mediului au un singur și repetabil efect; consecința acțiunii este știută cu siguranță
  - Nedeterminist: acțiunile pot avea efecte neprevăzute sau cu grade de incertitudine; consecința unei acțiuni nu poate fi dată cu certitudine
- **Episodic vs non-episodic**: într-un mediu episodic, performanța agentului și "experiența" sa se formează pe baza unor *episoade distincte de interacțiune* cu mediul, care sunt **independente** unul de altul (e.g. în simulări, în jocuri)

Mediul agenților

- **Static vs dinamic**
  - Static: proprietățile perceptibile de către agent ale mediului **nu** se schimbă în timp
  - Dinamic: proprietățile mediului se schimbă *în afara* influenței unor agenți
- **Discret vs continuu**
  - Discret: există un număr **finit** de acțiuni ale agenților și configurații perceptibile ale mediului – e.g. un joc de dame chinezești
  - Continuu: acțiunile au un spațiu continuu, iar stările mediului pot fi infinite (chiar dacă numărabile) – e.g. condusul unei mașini autonome, controlul unui swarm de drone

- Arhitecturi pentru agenți cognitivi
- Arhitecturi pentru agenți reactivi
- Modelul conceptual al agenților cognitivi

Arhitectura Belief, Desire, Intention (BDI)

- Privește un agent cognitiv ca pe un **sistem intențional** [Dennett, 1971], atribuindu-i în analogie cu o "psihologie umană":
  - **Convingeri (beliefs)**: setul de afirmații pe care un agent le **consideră adevărate** despre mediul în care operează și despre ceilalți participanți la mediu
  - **Dorințe (desires)**: setul de **obiective** (goals) pe care un agent *dorește* să le îndeplinească
  - **Intenții (intentions)**: **setul de acțiuni** (posibil deja parțial executate) care fac parte dintr-un **plan** construit pentru a atinge **un obiectiv** din setul celor dorite, pe care agentul **s-a angajat** să-l îndeplinească

Arhitectura Belief, Desire, Intention (BDI)

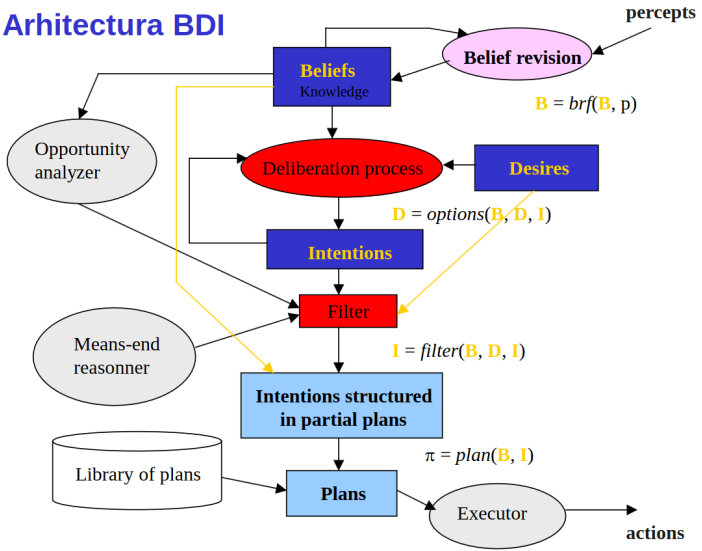
- Privește un agent cognitiv ca pe un **sistem intențional** [Dennett, 1971], atribuindu-i în analogie cu o "psihologie umană":
  - **Convingeri (beliefs)**: setul de afirmații pe care un agent le **consideră adevărate** despre mediul în care operează și despre ceilalți participanți la mediu
  - **Dorințe (desires)**: setul de **obiective** (goals) pe care un agent *dorește* să le îndeplinească
  - **Intenții (intentions)**: **setul de acțiuni** (posibil deja parțial executate) care fac parte dintr-un **plan** construit pentru a atinge **un obiectiv** din setul celor dorite, pe care agentul **s-a angajat** să-l îndeplinească

Cu ce ne ajută să atribuim *convingeri, dorințe* și *intenții* unui program software?

- Este o **manieră** de a discuta despre un **sistem complex**
- Util atunci când **avem nevoie** ca un **sistem complex de interacțiuni** să **ni se facă înțeles** (să se explice)

Arhitectura Belief, Desire, Intention (BDI)

- Privește un agent cognitiv ca pe un **sistem intențional** [Dennett, 1971], atribuindu-i în analogie cu o "psihologie umană":
  - **Convingeri (beliefs)**: setul de afirmații pe care un agent le **consideră adevărate** despre mediul în care operează și despre ceilalți participanți la mediu
  - **Dorințe (desires)**: setul de **obiective** (goals) pe care un agent *dorește* să le îndeplinească
  - **Intenții (intentions)**: **setul de acțiuni** (posibil deja parțial executate) care fac parte dintr-un **plan** construit pentru a atinge **un obiectiv** din setul celor dorite, pe care agentul **s-a angajat** să-l îndeplinească
- Este o **manieră** de a discuta despre un **sistem complex**
- Util atunci când **avem nevoie** ca un **sistem complex de interacțiuni** să **ni se facă înteles** (să se explice)


Arhitectura BDI
percepts
Belief revision
Beliefs
Knowledge
B = brf(B, p)
Opportunity analyzer
Deliberation process
Desires
D = options(B, D, I)
Intentions
Filter
Means-end reasonner
I = filter(B, D, I)
Intentions structured in partial plans
π = plan(B, I)
Library of plans
Plans
Executor
actions

Strategii de angajare (eng. commitment strategies)

- Opțiune (dintre dorințe) aleasă de agent ca intenție - agentul "s-a angajat" pentru acea opțiune
  - Angajare **oarbă** (eng. blind commitment)
  - Angajare **limitată** (eng. single minded commitment)
  - Angajare **deschisă** (eng. open minded commitment)

Bucla de control BDI

angajare oarba

```
B = B0
I = I0  D = D0
while true do
        get next perceipt p
        B = brf(B,p)
        D = options(B, D, I)
        I = filter(B, D, I)
        π = plan(B, I)
        while not (empty(π) or succeeded (I, B)) do
                α = head(π)
                execute(α)
                π = tail(π)
                get next perceipt p
                B = brf(B,p)
                if not sound(π, I, B) then
                        π = plan(B, I)        ← Reactivity, replan
        end while
end while
```

Bucla de control BDI

angajare limitata

```
B = B0
I = I0  D = D0
while  true  do
        get next perceipt p
        B = brf(B,p)
        D = options(B, D, I)
        I = filter(B, D, I)
        π = plan(B, I)
        while not (empty(π) or succeeded (I, B)  or impossible(I, B))  do
                α = head(π)
                execute(α)
                π = tail(π)
                get next perceipt p
                B = brf(B,p)
                if not sound(π, I, B) then
                    π = plan(B, I)
        end while
end while
```

Dropping intentions that are impossible or have succeeded

Reactivity, replan

Bucla de control BDI

angajare deschisa

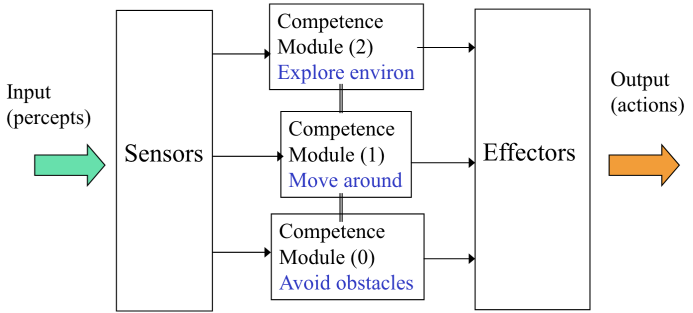
```
B = B0
I = I0  D = D0
while true do
    get next perceipt p
    B = brf(B,p)
    D = options(B, D, I)
    I = filter(B, D, I)
    π = plan(B, I)
    while not (empty(π) or succeeded (I, B) or impossible(I, B)) do
        α = head(π)
        execute(α)
        π = tail(π)
        get next perceipt p
        B = brf(B,p)
        if reconsider(I, B) then
            D = options(B, D, I)
            I = filter(B, D, I)
        π = plan(B, I)        ← Replan
    end while
end while
```

Arhitectura de subsumare

- Luarea deciziilor se face prin intermediul unor comportamente aferente fiecărui obiectiv (eng. Task Accomplishing Behavior – TAB)
  - Fiecare comportament (behavior) realizează **o acțiune**
  - Implementarea este o mapare de la situație →acțiune
- Mai multe comportamente pot fi activate in paralel

Arhitectura de subsumare

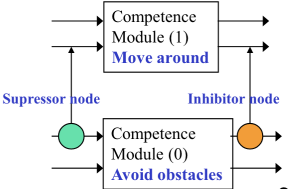
- Un TAB este reprezentat de un **modul de competenta**
- Fiecare modul de competenta executa un task simplu si opereaza **in paralel**
- Nivele *inferioare* ale modulelor de competenta au prioritate asupra modulelor *superioare*
  - Un modul inferior monitorizeaza si influenteaza intrarile si iesirile unui modul superior



M1 = **move around** while *avoiding obstacles* ⊃ M0

M2 = **explores the environment looking for distant objects of interests** while *moving around* ⊃ M1

- Incoroprarea functionalitatii unui c.m. subordonat de catre un c.m. superior se face prin noduri **supresoare** (modifica semnalul de intrare) si noduri **inhibitoare** (inhiba iesirea)



Pentru a înțelege cum putem pune problema **învățării unui agent** prin **interacțiunea cu un mediu** vom face următoarele abstractizări ale evenimentelor din mediu și ale acțiunilor agentului:

- Definim prin $S = \{s, s', s'', \ldots\}$ setul (posibil infinit) de **stări** instantanee în care se poate afla mediul
- Definim prin $A = \{a, a', a'', ...\}$ setul (posibil infinit) de **acțiuni** disponibile agentului, prin care acesta poate **modifica** starea sa internă sau pe cea a mediului

Atunci, definim drept episod de interacțiune cu mediul **o secvență** de perechi succesive de stări ale mediului și acțiuni executate de agent:

$$e : s_0, \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \ldots \xrightarrow{a_{n-1}} s_n$$

La nivel formal putem defini:

- Fie o secvență $(s_0, a_0, s_1, a_1, \ldots)$ ce reprezintă un episod al unui agent Ag într-un mediu $Env =< S, s_0, \tau >$ unde:
  - S - mulțimea de stări ale mediului
  - s_0 - starea inițială
  - τ - o funcție de transformare (**cu posibile efecte stocastice**)

- Atunci, definim recurența de stări ale mediului și acțiuni ale agentului ca:
  - $a_0 = Ag_{act}(s_0)$
  - $s_n = \tau((s_0, a_0, s_1, a_1, \ldots, a_{n-1}))$
  - $a_n = Ag_{act}((s_0, a_0, s_1, a_1, \ldots, s_{n-1}))$

Agenți care învață

- Scopul agenților este să îndeplinească **obiective** în favoarea unui utilizator sau aplicație
- **Agenților care învață** (singuri) vrem să le spunem **ce să facă**, dar nu și **cum să facă**

Provocare: cum informăm un agent astfel încât să învețe să distingă un **episod de interacțiune** *reușit* de unul *nereușit*?

Idee: asociem o noțiune de **utilitate**

Agenți care învață

- Scopul agenților este să îndeplinească **obiective** în favoarea unui utilizator sau aplicație
- **Agenților care învață** (singuri) vrem să le spunem **ce să facă**, dar nu și **cum să facă**

Provocare: cum informăm un agent astfel încât să învețe să distingă un **episod de interacțiune** *reușit* de unul *nereușit*?

Idee: asociem o noțiune de **utilitate**

- Fiecărei stări în parte: $u : S \to R$
  - Cum determinăm utilitatea unui întreg episod?
  - Cum distingem între utilități "mai apropiate" în timp față de cele îndepărtate în timp? (e.g. adăugăm o *atenuare*)

Agenți care învață

- Scopul agenților este să îndeplinească **obiective** în favoarea unui utilizator sau aplicație
- **Agenților care învață** (singuri) vrem să le spunem **ce să facă**, dar nu și **cum să facă**

Provocare: cum informăm un agent astfel încât să învețe să distingă un **episod de interacțiune** *reușit* de unul *nereușit*?

Idee: asociem o noțiune de **utilitate**

- Fiecărei stări în parte: $u : S \to R$
  - Cum determinăm utilitatea unui întreg episod?
  - Cum distingem între utilități "mai apropiate" în timp față de cele îndepărtate în timp? (e.g. adăugăm o *atenuare*)
- Unui întreg episod de interacțiune

Agenți care învață

- Scopul agenților este să îndeplinească **obiective** în favoarea unui utilizator sau aplicație
- **Agenților care învață** (singuri) vrem să le spunem **ce să facă**, dar nu și **cum să facă**

Provocare: cum informăm un agent astfel încât să învețe să distingă un **episod de interacțiune** *reușit* de unul *nereușit*?

Idee: asociem o noțiune de **utilitate**

- Fiecărei stări în parte: $u : S \to R$
  - Cum determinăm utilitatea unui întreg episod?
  - Cum distingem între utilități "mai apropiate" în timp față de cele îndepărtate în timp? (e.g. adăugăm o *atenuare*)
- Unui întreg episod de interacțiune

Probleme generale cu asignarea **utilităților**:

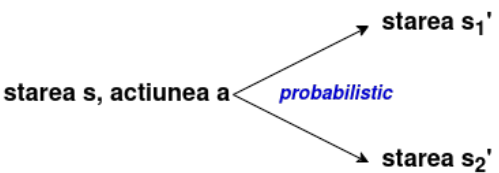
- De unde vin valorile?
- Cum gândim o problemă în termen de utilități obținute pe parcursul rezolvării ei?

Agenți care învață : MDP

- O clasă întreagă de metode de învățare a unui agent sau a unui sistem multi-agent se bazează pe modelarea mediului ca un Proces de Decizie Markov (eng. Markov Decision Process - **MDP**)

Caracteristică importantă:

- **Efect nedeterminist al execuției** unei acțiuni *a* într-o stare *s*. Exemple:
  - robotică: decide unde să te muți, dar actuarea funcționează parțial
  - alocare de resurse: decizi ce să produci, dar nu cunoști cererea pe piață a produsului tău
  - agricultură: decizi ce sa plantezi, dar nu cunoști schimbarea climei și influența asupra culturii

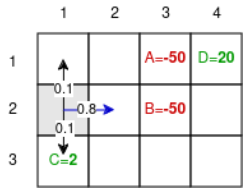


Agenți care învață : MDP

Un exemplu:

- Modelăm un teren vulcanic stâncos și alunecos ca un grid de 3 x 4, în care stările finale *A* și *B* sunt cu lavă vulcanică, iar stările finale *C* și *D* ne dau o priveliște frumoasă asupra vulcanului care a erupt.
- Începem deplasarea din celula gri. Mișcările permise sunt sus, jos, stânga, dreapta. Dacă ne "lovim" de marginea grid-ului, rămânem pe loc
- Terenul fiind alunecos acțiunea dorită (e.g. săgeata albastră) iese ca atare în 80% din cazuri. În restul, vom aluneca spre stânga sau spre dreapta.

Care sunt deciziile cele mai bune în fiecare stare pentru a ne maximiza **utilitatea** acestui mediu **în valoare așteptată**? (Care este **politica** de acțiuni?)



Agenți care învață : MDP

Ce înseamnă **valoare așteptată** a unei **variabile aleatoare**?

$$\mathbb{E}(X) = \sum_{x \in vals(X)} p(x) \cdot X$$

Exemplu: Un joc de zar cu șase fețe cu următoarele reguli

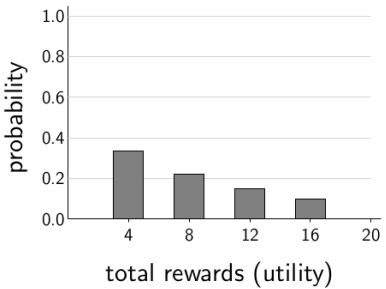
- Două acțiuni: *stop* și *continuă*
- Dacă alegem *stop* primim 10 RON și terminăm jocul
- Dacă alegem *continuă*:
  - primim 4 RON
  - aruncăm zarul: dacă iese 1 sau 2, jocul se oprește, altfel continuă

Care este câștigul maxim așteptat ((**utilitatea**)) în acest joc?

Agenți care învață : MDP

- Pentru **politica** de a juca mereu "continuă", valoarea așteptată este:

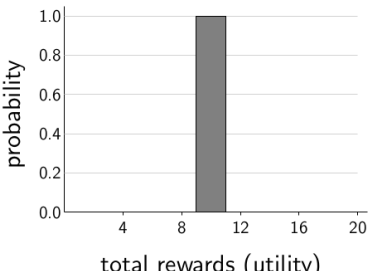
$$\frac{1}{3}(4) + \frac{2}{3} \cdot \frac{1}{3}(8) + \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3}(12) + \ldots = 12$$



Agenți care învață : MDP

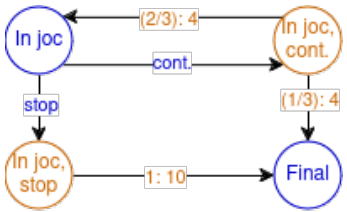
- Pentru **politica** de a juca mereu "stop", valoarea așteptată este:

$$1 \cdot 10 = 10$$



Agenți care învață : MDP

- Putem modela jocul de zar ca un graf cu noduri de tip **stare** și noduri de tip **șansă**.
- Pe muchii avem **probabilitatea** de a tranzita de la o **stare** la alta, precum și **recompensa** asociată acelei tranziții.



Agenți care învață : MDP

Mediul ca un Proces de Decizie Markov

- **Stări** S, $s_{start} \in S$
- **Acțiuni** A: setul de acțiuni posibile în problema modelată
- $\mathbf{T}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$: funcția de tranziție – modelează probabilitatea $p(s'|s, a)$ de a ajunge în starea s', dacă se execută acțiunea a în starea s
- $\mathbf{R}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$: **recompensa** aferentă tranziției (s, a, s') – valoare care ajută la calcularea **utilității** unui **episod de interacțiune** cu mediul
- $Final(s)$: funcție booleană care stabilește dacă o stare s este una care **termină** un episod de interacțiune în mediu
- **Factor de atenuare (discount)**: $0 \leq \gamma \leq 1$ – factor ce permite *atenuarea* influenței recompensei din viitor în favoarea celei din prezent

Agenți care învață : MDP

Caracteristică funcție de tranziție:

$$\forall s, \forall a : \sum_{s' \in S} T(s, a, s') = 1$$

Pentru exemplul jocului de zar:

s	a	s'	T(s,a,s')
in joc	stop	final	1
in joc	cont.	in joc	2/3
in joc	cont.	in joc	1/3

Stările **succesor** s' ale unei stări s sunt acelea pentru care $T(s, a, s') > 0$

Agenți care învață : MDP

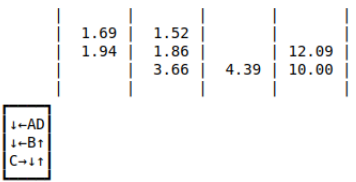
Ce este **soluția** unui MDP?

- O cale (secvență de acțiuni) determinată de o **politică**

O **politică** $\pi : S \rightarrow A$ este o funcție care mapează fiecărei stări $s \in S$ o acțiune $a \in A$

- În fiecare stare va exista **o cea mai bună** acțiune care se ia de fiecare dată când suntem într-o stare s
- **Proprietatea Markov** a MDP-ului: o stare încapsulează toată informația asupra situația instantanee a mediului, astfel că tranziția de la o stare curentă la una succesoare depinde doar de starea curentă și acțiunea luată ($p(s'|s, a)$)

Obiectivul unui agent într-un MDP: **găsirea politicii optime** π^*



Mulțumesc!



Feedbackul este binevenit!


Dennett, D. C. (1971).
Intentional systems.
The Journal of Philosophy, 68(4):87–106.